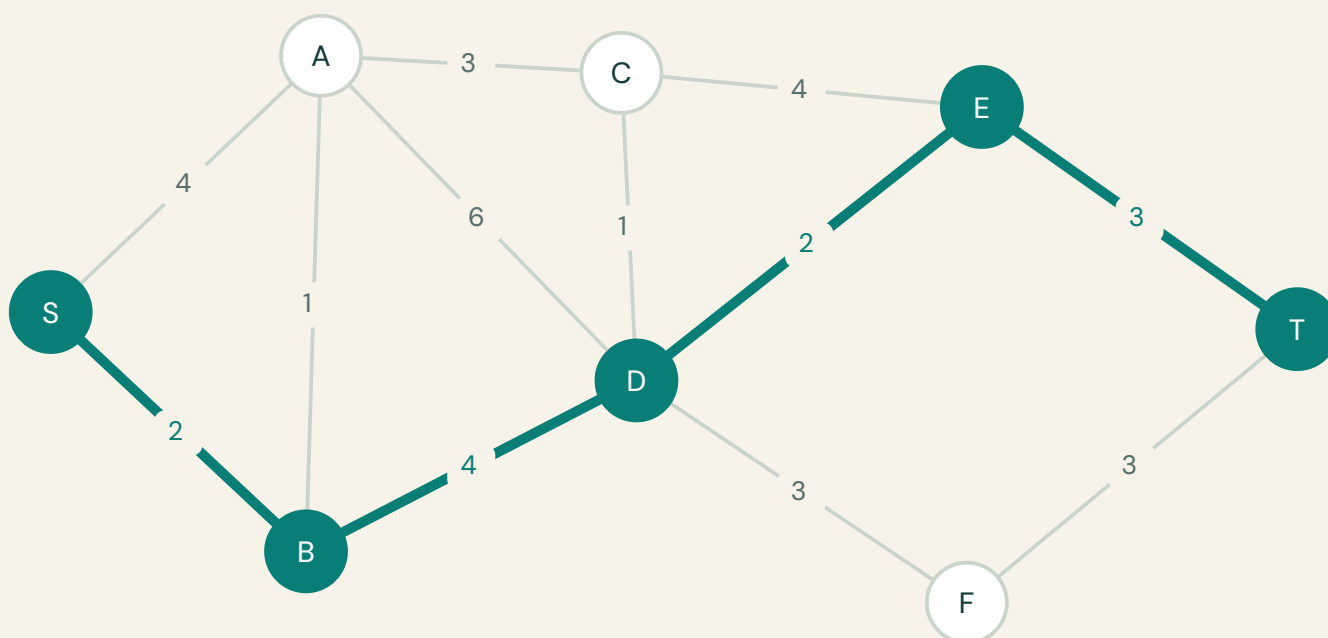


Una ruta. Cada decisión.

Dijkstra orientado a objetos, con costos exactos y evidencia reproducible.

VÉRTICE

Laboratorio de rutas mínimas



RUTA CALCULADA

S > B > D > E > T

COSTO 11

Grafo didáctico incluido en `examples/desvio.json`. El resultado se genera con el mismo núcleo Python de la entrega; los costos no representan distancias medidas de una calle.

Del lineamiento a la evidencia

Referencia: Bootcamp SDV.pdf, página 3. Perfil Software: Dijkstra y POO.

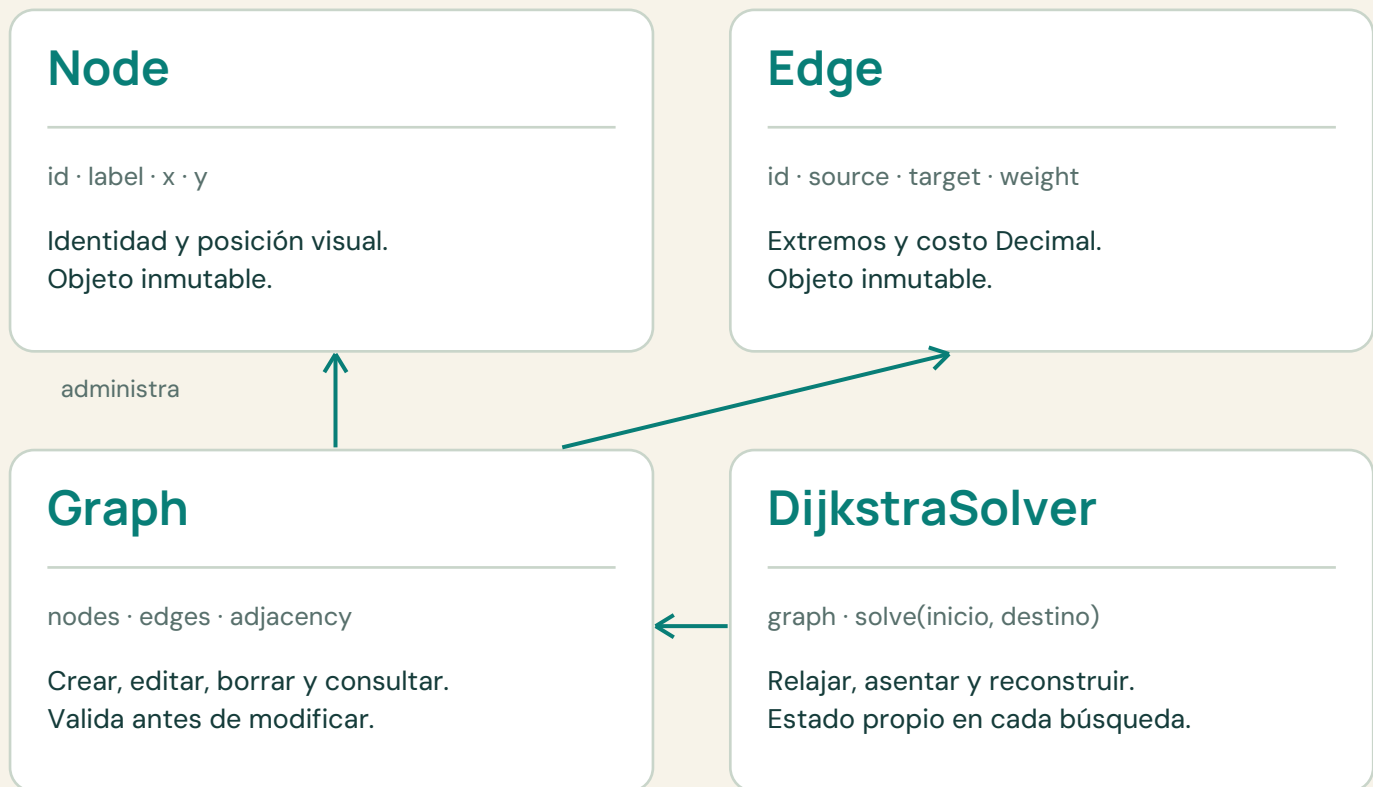
REQUISITO	IMPLEMENTACIÓN	DÓNDE COMPROBAR
Dijkstra desde cero	DijkstraSolver: relajaciones, cola, predecesores y ruta propios.	vertice/dijkstra.py
Objetos del sistema	Node, Edge y Graph; edición validada y adyacencia consistente.	graph.py; API.md
Crear y modificar	Terminal editable y editor web; inicio y destino explícitos.	cli.py; interfaz
Clases documentadas	Atributos, métodos, responsabilidades y representación visual.	API.md; p. 3
Pruebas distintas	Casos nombrados, grafos con semilla y oráculo independiente.	p. 6; evidencia JSON
Ausencia de ruta	no_path, rutas vacías y costo null; sin inventar un costo cero.	tests; ejemplos
Decisiones y operación	Trazas reales, argumento de corrección y complejidad explícita.	pp. 4-5; ALGORITMO.md

VALOR ADICIONAL DENTRO DEL ALCANCE

La edición reversible, la visualización de eventos y la auditoría ayudan a explicar y verificar el reto. La GUI es opcional en el documento; aquí complementa la implementación Python. No se presenta como ejecución en un vehículo.

Objetos y responsabilidades

El grafo conserva las reglas; el solucionador calcula; la interfaz representa el resultado.



ADYACENCIA DEL EJEMPLO

S A : e1 | B : e2

B A : e3 | D : e6 | S : e2

T E : e11 | F : e12

Estructura: nodo → vecino → ID de conexión. En no dirigido se conservan ambas orientaciones; un bucle propio aparece una vez. Borrar un nodo retira sus conexiones. Una edición inválida conserva el grafo anterior.

La traza explica el resultado

Ejemplo “Un mejor desvío”: la conexión directa no siempre produce el menor costo.

8

nodos asentados

9

mejoras aplicadas

2

entradas antiguas
descartadas

PASO	OPERACIÓN	NODO / CONEXIÓN	COSTO
0	Iniciar	S	0
1	Asentar	S	0
3	Mejorar	S > A	sin dato > 4
5	Mejorar	S > B	sin dato > 2
6	Asentar	B	2
8	Mejorar	B > A	4 > 3
18	Descartar	A	4 ya reemplazado por 3
29	Mejorar	D > E	10 > 8
42	Terminar	T	11

Selección de 9 eventos de una traza real de 43. Los números de paso conservan su posición original. “Sin dato” significa que el nodo todavía no tenía una ruta descubierta.

La mejora clave: S descubre A con costo 4; al procesar B aparece una ruta a A de costo 3. La entrada antigua de costo 4 se descarta después, sin volver a expandir A.

Por qué el mínimo es correcto

Premisas: conexiones válidas, pesos no negativos y grafo estable durante cada búsqueda.

01

Cada costo representa una ruta

El origen empieza en cero. Relajar añade una conexión real a una ruta conocida; nunca inventa una distancia menor que el óptimo.

02

El menor vigente queda resuelto

Una ruta más barata cruzaría antes por un nodo pendiente de costo menor. Ese nodo habría salido primero de la cola. La no negatividad hace válida esta contradicción.

03

Los predecesores reconstruyen

Un predecesor está asentado antes que su sucesor. La cadena retrocede en ese orden, no forma ciclos y termina en el inicio.

21 cifras necesarias · 40 cifras disponibles

Con 500 nodos, peso máximo 10^{12} y seis decimales, una suma candidata usa como máximo 15 cifras enteras + 6 fraccionarias. El contexto Decimal explícito representa esas sumas sin redondear.

PARAR TEMPRANO EXIGE UNA DISTINCIÓN

Se termina al asentar el destino. Solo los nodos de settled tienen distancia definitiva. Si $A \rightarrow B = 1$, $A \rightarrow C = 9$ y $B \rightarrow C = 1$, buscar $A \rightarrow B$ deja a C con costo tentativo 9; su óptimo global sería 2.

Si se agota la cola sin asentar el destino, no hay ruta. Inicio igual a destino devuelve costo cero. El argumento completo, las cotas y los casos límite están en ALGORITMO.md.

Contraste independiente

Bellman-Ford con enteros en micro-unidades comprueba los costos de Dijkstra.

38

métodos independientes

160

grafos con semilla

631

consultas con semilla

CATEGORÍA	MÉTODOS	ESTADO
Edición e invariantes	8	Aprobados
Algoritmo y oráculo	14	Aprobados
Entradas y números	16	Aprobados

Además: 18 casos nombrados. Las consultas son ejecuciones dentro de los métodos; no se suman como si fueran nuevas pruebas unitarias. El oráculo usa otra estrategia y no reutiliza la relajación de Dijkstra.

Dominio exhaustivo acotado: 729 grafos dirigidos de tres nodos y 6561 consultas. Cada una de las seis conexiones posibles toma ausencia, costo 0 o costo 1. No se extrapola exhaustividad a grafos mayores.

Se revisaron 59070 eventos por invariantes. Floyd-Warshall contrastó el propio oráculo. Las pruebas detectaron 9 defectos introducidos en copias aisladas; no es una garantía universal.

Evidencia: independent.json, cases.json y seeded_graphs.json. Fecha, entorno, semillas y huellas SHA-256 acompañan los resultados. El estado mostrado requiere que coincidan las huellas del código actual.

Algoritmo y recorrido completo

Medianas en milisegundos. La validación y la serialización también cuestan tiempo.

Caso: nodos / conexiones

■ Algoritmo

■ JSON a JSON

Cadena 50 / 49

0.53 / 2.22 ms

Cadena 200 / 199

2.04 / 9.03 ms

Cadena 500 / 499

5.48 / 23.34 ms

Dirigido 500 / 4000

11.51 / 103.48 ms

Malla 400 / 760

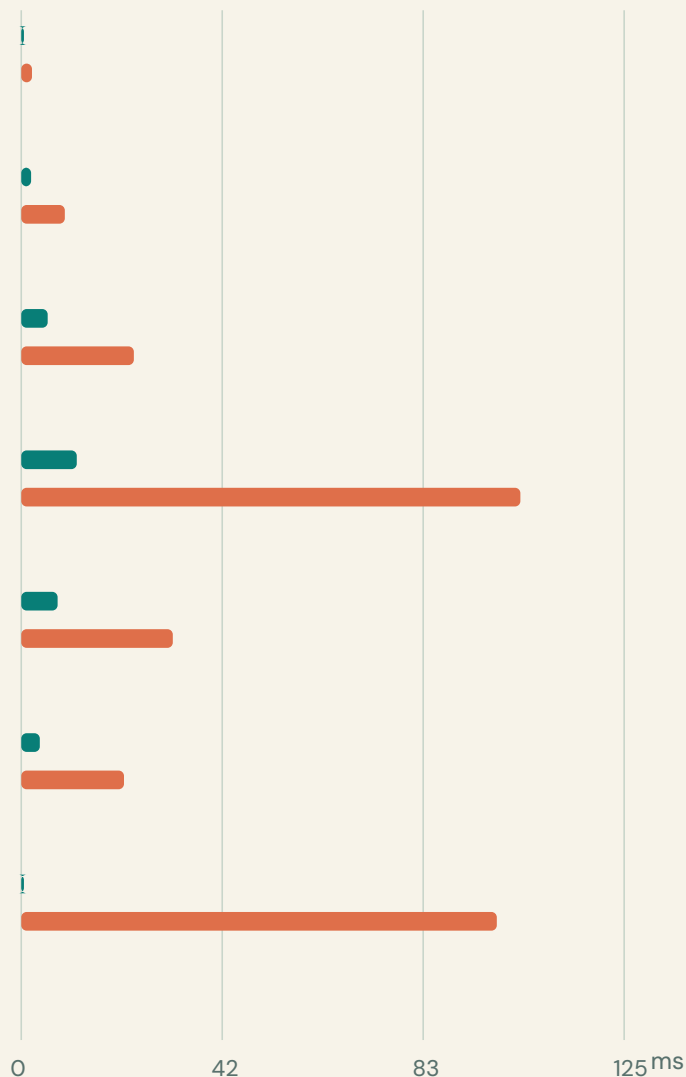
7.53 / 31.42 ms

Sin ruta 500 / 498

3.84 / 21.30 ms

Inicio = destino 500 / 4000

0.15 / 98.60 ms



Método: 2 calentamientos y 9 repeticiones por variante. Siete cargas, con y sin traza; aquí se muestra sin traza. "Algoritmo" excluye construir el grafo; "JSON a JSON" incluye validar, construir, resolver y serializar.

CPython 3.11.9 · Windows · AMD64 Family 25 Model 80 Stepping 0, AuthenticAMD. Semilla 20260912 · reloj perf_counter_ns. Una computadora; sin garantía de tiempo real. Con nueve muestras, el percentil95 por rango más cercano coincide con el máximo. Huellas de cálculo comprobadas.

La primera versión no bastaba

Se conservaron fallos anteriores y se añadieron regresiones para comprobar las correcciones.

HALLAZGO	CONSECUENCIA	CAMBIO
Números JSON extremos	Excepción o trabajo cuadrático.	Límite temprano y recorrido lineal.
Contexto decimal externo	Podía cambiar la aritmética.	Todos los atributos explícitos.
JSON convertido antes de validar	Pérdida silenciosa de cifras o claves.	Validar el texto original en Python.
JSON amplio inválido	Memoria elevada antes del rechazo.	Límites de estructura previos.
Cancelación compartida	Interrumpía solicitudes ajenas.	FIFO y cancelación independiente.
Etiquetas y límite del archivo	Espacios perdidos y emoji cortados.	Datos intactos y límite de 2 MiB.
Salida del empaquetador	Podía sobrescribir un archivo fuente.	Salida externa y escritura temporal.
Cabeceras y rangos HTTP	Ambigüedad y semántica incorrecta.	Validación estricta y regresiones.
Zoom sobre grafos densos	Los nodos tapaban los controles.	Recorte al viewport del SVG.

LA REVISIÓN DEJA LÍMITES VISIBLES

Aprobar pruebas no demuestra ausencia de todos los defectos. Los costos son abstractos, la legibilidad de grafos densos tiene límites y una medición local no representa otros equipos. La defensa debe poder explicar y modificar el código.

AUDITORIA.md enlaza los registros anteriores y posteriores, incluidas revisiones de interfaz, video y entrega. No se asigna una calificación oficial ni una aprobación de un evaluador externo.

Comprobar cada superficie

Un núcleo correcto no demuestra por sí solo que el navegador o la publicación funcionen.

COMPONENTE	ESTADO	EVIDENCIA
API local HTTP	Verificado	26 métodos. Solicitudes reales a localhost con fixtures aislados.
Terminal y arranque	Verificado	12 métodos. CPython 3.10.11, 3.11.9, 3.12.14, 3.14.7. Misma suite; no son casos adicionales.
Python en navegador	Verificado	29 controles reales: resultados y trazas de 18 casos coinciden con Python nativo. Importación y cola verificadas.
Editor y estabilidad	Verificado	54 controles funcionales, 22 adversos y 192 de zoom. Ensayo completo: 12,16 min, 408 acciones; sin fallos.
Sitio y video	Verificado	Primera publicación: 31 controles anónimos. Video: 20 controles. El ZIP final lleva comprobantes separados en Releases.

USO PREVISTO

Abrir el proyecto, elegir un ejemplo, seleccionar inicio y destino y calcular. Después, editar un costo y observar cómo cambia la ruta. La reproducción recorre eventos; no modifica el resultado del algoritmo.

Una respuesta calculada para un grafo anterior debe descartarse tras editar. El modo web ejecuta el paquete Python en un worker; la cancelación y la identidad del worker tienen pruebas separadas de la paridad matemática.

Código, evidencia y límites a la vista

Núcleo y terminal: Python 3.10 o posterior, solo biblioteca estándar.

DESDE LA CARPETA DEL PROYECTO

```
python -m vertice.cli validate examples/desvio.json
python -m vertice.cli solve examples/desvio.json -s S -t T --json
python -m unittest discover -s tests -p test_core.py
python scripts/verify_independent.py
python scripts/benchmark.py
node --test tests/engine_queue.mjs
```

Añade `--trace` a `solve` para exportar cada evento. En Windows, `Iniciar.cmd` abre la aplicación local. Las herramientas de PDF, video y verificación del navegador son dependencias de producción de la entrega, no del algoritmo.

COMPLEJIDAD DECLARADA

Tiempo $O((V + E) \log(V + E + 1))$

Memoria $O(V + E)$

La cota incluye vecinos ordenados, cola con entradas antiguas y trazas. La precisión numérica está acotada. El detalle por operación está en `ALGORITMO.md`.

FUENTES PRIMARIAS Y ARCHIVOS DE EVIDENCIA

Bootcamp SDV.pdf, p. 3	Documento suministrado; perfil Software.
Dijkstra (1959), problema 2	https://ir.cwi.nl/pub/9256/9256D.pdf
Python: Decimal	https://docs.python.org/3/library/decimal.html
Python: heapq	https://docs.python.org/3/library/heapq.html
Evidencia de ejecución	evidence/verification/; semillas y SHA-256 en JSON.